

AIops Module 1 Assignment Report

Aakash, DA24B028, IIT Madras , Repo: <https://github.com/AakashAadhithya/aiops-assignment1> Partner: Arnav, DA24B027

Q1 Technical Debt Diagnosis (10 marks)

(a) The delivery-time rounding change breaking the unrelated favourites feature is **Entanglement (CACE)** – there’s no clean boundary between the two features, so a change meant for one silently altered the other’s behaviour. This is the “changing anything changes everything” problem: nothing crashed, nothing threw an error, the model just quietly got worse somewhere nobody was looking. (b) Marketing silently reading the model’s raw output table is **Undeclared Consumers** – the ML team has no visibility into who depends on that table, so a routine schema change or retrain could break marketing’s dashboard with zero warning. (c) 14 undocumented shell scripts with no orchestration tool is **Glue Code / Pipeline Jungles** – there’s no single source of truth for how data actually flows end to end, which makes onboarding and debugging painful. Mitigation (for b): replace direct table access with a governed interface – the **MLflow Model Registry** with defined stages, or a feature store – so consumers must formally subscribe to a declared version instead of reading raw internal state. Full write-up: README.md §Q1.

Q2 MLflow Experiment Comparison (15 marks)

Swept `hidden_size` ∈ {32, 64, 128} and `lr` ∈ {0.001, 0.01} (6 runs, MLP replacing RandomForest, MNIST replacing IRIS, per the assignment). All 6 results:

hidden_size	lr	train_loss	val_accuracy
128	0.01	0.036	0.945
128	0.001	0.020	0.948 (best)
64	0.01	0.037	0.933 (lowest)
64	0.001	0.040	0.945
32	0.01	0.017	0.937 (lowest loss)
32	0.001	0.087	0.941

Best run: `hidden128-lr0.001`. Overfitting evidence: `hidden32-lr0.01` has the sweep’s lowest `train_loss` (0.017, an almost perfect fit on training data) but only mid-pack `val_accuracy` (0.937) – worse than several runs with higher `train_loss`, meaning it memorized the training set without generalizing as well. Averaging by group, `hidden_size=128` gives 0.9465 `val_accuracy` vs. ≈ 0.939 for 64/32, while `lr=0.001` gives 0.9447 vs. 0.9383 for `lr=0.01` – `hidden_size` shows the more consistent effect (128 wins at both learning rates), though all 6 runs stay within a tight 93.3–94.8% band, so neither hyperparameter dominates dramatically here. Logging code: `train.py` lines 67–89. Screenshots: `screenshots/q2_comparison_1.png`, `q2_comparison_2.png`.

Q3 DVC Data Versioning & Rollback (10 marks)

Initialized DVC with an **SSH remote** (`ssh://localhost/home/aakash/dvcstore`, via the `dvc-ssh` plugin) rather than a local one, as required. `v1`: 1800 files + a generated `filenames.csv` (1801 lines incl. header), pushed (1802 files transferred). `v2`: added `new-labels.zip` → 2800 files, `filenames.csv` regenerated (2801 lines, matching the assignment’s target exactly), committed with a descriptive message and pushed (1002 new files transferred – DVC only pushes what changed relative to `v1`). Rollback via `git checkout <v1-commit> + dvc checkout` restored exactly 1800 files / 1801 CSV lines, confirmed by `dvc status` reporting “Data and pipelines are up to date.” Key lesson: `git checkout` alone only moves the `.dvc` pointer back in time – it doesn’t touch data on disk; only `dvc checkout` pulls the matching content from cache. Proof: `screenshots/q3_rollback.png`.

Q4 Capstone: End-to-End Reproducibility Drill (15 marks)

Partner A (me): trained the MLP with the best Q2 config (`hidden_size=128`, `lr=0.001`, `seed=42`, 20 epochs; run `capstone-q4`, `val_accuracy` ≈ 0.948 –0.951), logging params, per-epoch metrics, seed, a `git_commit` tag, and the model artifact to MLflow. The training code and the `v2 data.dvc` pointer already shared a single Git commit (`b360abc`), satisfying the “code + data versioned together” requirement automatically. Registered the model as `mnist-mlp-classifier v1` and transitioned it to **Staging** (`register_model.py`).

Partner B (Arnav, DA24B027): we met on a shared mobile hotspot; a bridged VM network attempt failed (hotspots often reject bridged traffic), so we used a `socat` TCP relay on my Mac forwarding to the VM’s SSH/MLflow ports – a real instance of the environment friction Lecture 1 calls the “production gap.” He cloned the repo, checked out `b360abc`, pointed DVC at my relay, and ran `dvc pull` (2802 files fetched). He recreated the environment with `conda` (no `mamba` available) and reran the identical command against my MLflow server.

Result: all 20 logged epochs matched exactly (e.g. epoch 20: `train_loss=0.0203`, `val_accuracy=0.948` on both sides) – strong evidence that the pinned code, environment, and data genuinely reproduce the same result on a different machine. He logged a confirmation note on the reproduced run stating this. Evidence: `screenshots/q4_partner_terminal.png`, `q4_mlflow_note.png`.

Overall takeaway: the questions build on each other in practice – Q2’s best config fed into Q4’s run, and Q3’s SSH-remote pattern was reused for Q4’s cross-machine transfer. The exact epoch match confirms Lecture 1’s reproducibility protocol works end to end.